TSV-TOD: Detecting Order-Dependent Flaky Tests in the Modern TypeScript Projects

Jeff Zou
Department of Computer Science
University of Illinois Urbana-Champaign
Urbana, IL, USA
jizezou2@illinois.edu

***Abstract*—Flaky tests are a significant burden on software development, wasting computational resources and undermining developers' confidence in testing. A major category of flakiness is Order-Dependent (OD) tests, defined as tests that pass or fail depending on the order of execution. While tools exist for Java (e.g., iFlakies, NonDex) and Python (e.g., iPFlakies), the modern JavaScript/TypeScript ecosystem, especially the emerging Vitest testing framework, lacks dedicated tooling. This paper introduces TSV-TOD (TypeScript Vitest Test Order-dependency Detector), a tool designed to detect OD tests in Vitest projects. TSV-TOD was evaluated on 14 open-source TypeScript projects, identifying 48 order-dependent tests across 4 projects. The findings suggest that while OD tests are less prevalent in JavaScript than in other languages, they remain a non-trivial issue, often caused by shared mutable state and improper cleanup.**

***Index Terms*—Software Testing, Flaky Tests, Order-Dependent Tests, JavaScript, TypeScript, Vitest**

I. Introduction

Regression testing is a fundamental technique for modern software engineering to ensure that new changes do not break backward compatibility. However, flaky tests—tests that show inconsistent results when performed against the same code version—often weaken the trustworthiness of these tests. Flaky tests are a pervasive problem, with studies showing they can account for a significant portion of build failures in large-scale systems [1]. They waste developer time, delay releases, and erode trust in the testing process.

A common type of flake tests is the *Order-Dependent (OD)* test. An OD test passes in one execution order but fails when run in another order. This behavior typically arises from tests that modify a shared state (e.g., global variables, database entries, file system) without resetting it, causing a subsequent test to fail.

While existing research has addressed OD tests in Java (e.g., iFlakies [2], NonDex [3]) and Python (e.g., iPFlakies [4]), the JavaScript ecosystem has received less attention regarding OD flakiness. This is surprising given JavaScript's dominant position in web development. Previous work, such as JS-TOD [6], focused on the Jest framework. However, the community is quickly shifting towards *Vitest*, a next-generation testing framework powered by Vite, known for its speed and native TypeScript support.

TSV-TOD is a tool specifically designed to detect order-dependent flaky tests in Vitest projects. TSV-TOD leverages Vitest's custom sequencer API to permute test execution orders, systematically exposing hidden dependencies. We applied TSV-TOD to a dataset of 14 popular open-source projects and analyze the prevalence and causes of the detected OD tests.

II. Background

Vitest has emerged as a leading testing framework for JavaScript and TypeScript, offering compatibility with the Jest API while providing significant performance improvements through its integration with Vite. According to the State of JS 2024 survey, Vitest has seen a huge rise in adoption and retention, rapidly becoming the de facto standard for new projects. Unlike Jest, which uses CommonJS (ESM was only added recently as an experimental feature), Vitest leverages native ES modules (ESM) and takes advantage of Vite's fast module resolution and hot module replacement capabilities. This results in significantly faster test execution times, especially for large codebases.

Order-dependent flakiness is a specific category of test failure where the outcome of a test depends on the execution order of the test suite. In other words, an order-dependent test is one that passes when run in a specific order but fails when the order is changed. This behavior is typically caused by a "polluter" [5] test that modifies a shared state (such as a global variable, database entry, or file system) without properly resetting. When a subsequent "victim" test relies on the default state, it fails because the polluter has left the environment in an unexpected condition. Conversely, a test might rely on a state set by a prior test (a "cleaner") and fail when run on its own.

III. Methodology

A. TSV-TOD Implementation

TSV-TOD is implemented as a command-line interface (CLI) tool that wraps the Vitest runner. It operates by defining a custom `TestSequencer` that intercepts the list of tests discovered by Vitest and reorders them based on a specified configuration before execution.

The tool supports several permutation strategies, inspired by iFlakies and JS-TOD:

- **Original:** Executes tests in the default order defined by the file structure.

- **Random-Group:** Randomizes the order of task groups. In Vitest, consecutive tasks (tests or suites) with the same concurrency mode (sequential or concurrent) form a group. This strategy maintains the relative order of tests within each group.
- **Random-Test:** Randomizes the order of individual tests within their respective task groups, in addition to randomizing the order of the groups themselves.
- **Reverse-Group:** Reverses the order of task groups.
- **Reverse-Test:** Reverses the order of individual tests within their respective task groups.

Crucially, TSV-TOD respects Vitest's concurrency model. Vitest groups tests into concurrent and sequential tasks. TSV-TOD ensures that permutations do not violate these grouping constraints, preventing invalid execution states where concurrent tests are forced to run sequentially or vice-versa.

B. Detection Process

TSV-TOD detects order-dependent tests through an automated process:

1) **Isolation:** For each test file execution, the tool creates a unique temporary directory. This directory holds intermediate data and ensures that the detection process does not pollute the source project.
2) **Configuration Generation:** A dedicated `vitest.config.ts` is generated within the temporary directory. This configuration dynamically imports the project's original settings (from `vite.config.ts` or `vitest.config.ts`) while injecting TSV-TOD's custom sequencer and runner logic.
3) **Execution:** The tool spawns a Vitest process using the generated configuration. The specific permutation strategy (e.g., `random-test`) and a random seed are passed to the runner via environment variables.
4) **Result Collection:** Upon completion, the custom runner serializes the execution order and test results into a JSON report stored in the temporary directory. TSV-TOD parses this report to identify failures and reports the specific seed required to reproduce them.

IV. Evaluation

A. Dataset

TSV-TOD is evaluated on 14 open-sourced TypeScript projects that use Vitest. The projects vary in size and domain, ranging from frontend focused libraries (e.g., `vueuse`, `unocss`) to developer tooling (e.g., `eslint-config`, `tinylibs/tinyspy`).

B. Results

Table I summarizes the results of this evaluation. A total of 48 order-dependent tests across 4 of the 14 projects were detected.

TABLE I
Order-Dependent Tests Detected by TSV-TOD

Project	OD Files	OD Tests	Total Files
antfu/unocss	4	13	87
rolldown/tsdown	4	15	12
vueuse/vueuse	3	3	210
paritytech/asset-transfer-api	1	17	92
antfu/eslint-config	0	0	3
antfu/eslint-plugin-command	0	0	21
aooiuu/any-reader	0	0	2
davelosert/vitest-coverage-report-action	0	0	14
elk-zone/elk	0	0	6
eslint-stylistic/eslint-stylistic	0	0	123
mayneyao/eidos	0	0	29
slidevjs/slidev	0	0	5
sxzz/unplugin-utils	0	0	2
tinylibs/tinyspy	0	0	3

V. Discussion

Our results indicate a relatively low prevalence of order-dependent tests in the evaluated JavaScript/TypeScript projects compared to similar studies in other ecosystems. This observation aligns with findings from the JS-TOD study [6], which also reported a lower frequency of OD tests in Jest projects.

This phenomenon is likely caused by a number of factors:

- **Test Isolation:** By default, test files in Vitest and Jest are executed in parallel using worker threads, and within files, developers often group related tests using `describe` blocks, which helps contain side effects. This model also makes shared inter-file dependencies more easily caught during development.
- **Programming Paradigms:** The JavaScript ecosystem heavily utilizes functional programming patterns and closure-based scoping as opposed to shared mutable class member variables in OOP-centered programming languages such as Java. These practices naturally reduce reliance on shared mutable state, which is a primary cause of order dependency.
- **Framework Behavior:** Vitest and Jest run test cases within a file sequentially by default. This deterministic execution order masks potential dependencies that only surface when the order is explicitly randomized. The default behavior of popular frameworks also leads to fewer observed OD tests unless specific permutation strategies are applied.

OD tests do exist and can be dangerous despite their lower prevalence. For example, in the `antfu/unocss` project, we confirmed that the detected failures were caused by shared mutable state that was not properly reset between tests. This shows that, despite being less common, shared state and inadequate cleanup are the same underlying causes of OD tests in JavaScript as they are in other languages.

VI. Conclusion

TSV-TOD, a tool for detecting order-dependent flaky tests in the Vitest ecosystem. The evaluation on 14 projects identified

48 OD tests, demonstrating that while less rampant than in other languages, order dependency remains a valid concern in modern JavaScript development. Future work will focus on automatically identifying the specific polluter-victim pairs and generating patches to isolate these tests.

REFERENCES

[1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014, pp. 433–444.

[2] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFlakies: A framework for detecting and fixing flaky tests," in *Proceedings of the 41st International Conference on Software Engineering*, 2019, pp. 545–555.

[3] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, "NonDex: A tool for detecting and debugging wrong assumptions on Java API specifications," in *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2016, pp. 993–997.

[4] R. Wang, Y. Chen, and S.-C. Cheung, "iPFlakies: A framework for detecting and fixing Python order-dependent flaky tests," in *Proceedings of the 44th International Conference on Software Engineering*, 2022.

[5] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *Proceedings of the 12th IEEE International Conference on Software Testing, Verification and Validation*, 2019, pp. 312–322.

[6] N. Hashemi, A. Tahir, S. Rasheed, A. Shi, and R. Blagojevic, "Detecting and Evaluating Order-Dependent Flaky Tests in JavaScript," *arXiv preprint arXiv:2501.12680*, 2025.